SIMATS ENGINEERING

CSA1023 – SOFTWARE ENGINEERING

ASSIGNMENT 1

CO1: Apply agile methodologies and project management techniques such as scale and lean to effectively manage software projects using industry-standard tools like Jira.

Student Name		**Register Number**	
Department / Section	BIOTECHNOLOGY	**Academic Year / Semester**	2026-2027
Faculty Name	Karpagam V	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission		**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

	Task Title	Max	Marks Awarded	Remarks
1	Algorithm Design and Grading Comparison	4		
2	Pipeline Design and Operation	5		
3	Image Representation Analysis	4		
4	Connectivity Analysis	4		
5	Image Acquisition Analysis	5		
6	Sampling Analysis	5		
7	Quantization Analysis	4		
8	Image Formation Analysis	5		
9	Implementation and Performance Evaluation	6		
10	Front-End Adaptation Challenge	8		

	TOTAL	**50**		

OVERALL REMARKS:

PREKSHA PRAKASH		
Student Signature	**Faculty Signature**	**Program Director**

SOFTWARE ENGINEERING
Assignment 1

SIMATS ENGINEERING

CSA10 – SOFTWARE ENGINEERING

ASSIGNMENT 1

CO1: *Apply agile methodologies and project management techniques such as scale and lean to effectively manage software projects using industry-standard tools like Jira.*

Assignment Title: Design, Simulate and Validate an Agile Delivery Strategy for the ITSM Platform

Nature of Assignment: Individual

Duration: One week

Total Marks: 50

Submission Date: _____

Scenario — Agile Delivery Planning for the ITSM Platform

An enterprise plans an integrated ITSM platform to reduce delayed incident response, repeated handovers and unclear ownership. Service desk analysts prioritize intake and SLA visibility, SREs prioritize telemetry correlation, and managers require auditable approvals and developer issue links. An eight-member team must select a feasible six-week release despite uncertain capacity, limited stakeholder time and delayed integration access. Create and validate an Agile delivery strategy connecting these priorities to a justified backlog, team responsibilities, Lean flow controls and sprint goals. Revise the strategy when business units, staffing and coordination needs change, while protecting a complete incident-resolution journey.

Planning conditions for the representative release are stated below.

Observation	Scenario planning condition
Operational scope	Incident intake, triage, telemetry, approvals, problem investigation and DevOps links
Users	Administrator, SRE, analyst, CAB reviewer, manager and employee
Reference data	Synthetic incident records and controlled telemetry events
Integration constraint	External issue tracker may be unavailable; preserve local incident records and retry safely
Validation need	Verify priority, status, SLA, authorization, persistence and audit behaviour

The organization proposes an AI-assisted IT Service Management and Incident

Resolution Platform. Its intended modules are an operations dashboard, assisted triage and diagnostics, incident lifecycle management, infrastructure telemetry, a service request catalog, change approval, problem investigation and a DevOps integration hub. The system should connect an alert or report to a tracked incident, authorized investigation, a justified change, deployment verification and closure.

An eight-member cross-functional team plans an initial release within six weeks using three two-week sprints. There is no established velocity. Test access to the external issue-tracking interface begins in week three. Service desk and change-review stakeholders can attend clarification and review meetings for two hours per week. These are assignment planning assumptions.

Principal users are enterprise administrators, SRE or incident commanders, service desk analysts, change advisory board reviewers, department managers and standard employees. They need different permissions. Analysts require useful diagnostic suggestions; managers require auditable approvals; engineers require traceable links to development issues and releases. Automation must preserve human responsibility for prioritization, approval and operational changes.

Define workload, service-level policies and acceptance thresholds as explicit planning assumptions. Dashboard values, diagnostic scores and screenshots are illustrative evidence of displayed behaviour, not proof of production accuracy, performance, live monitoring or secure authorization. Verify claims through repeatable checks. Distinguish deterministic keyword rules from trained machine learning and label simulations and proposed extensions accurately.

Design Challenge:

Management requests all eight modules in the initial release. Service desk analysts prioritize incident intake, assignment and SLA visibility; SREs prioritize telemetry-to-incident correlation; managers prioritize service and change approvals. Integration access and limited review time constrain delivery. A broad release may leave every workflow incomplete, while a narrow dashboard-only release may fail to improve incident resolution.

Design, simulate and validate an Agile project delivery strategy using a project-management tool of your choice that produces a coherent and achievable first release under these conditions.

Develop alternative delivery approaches and create a justified strategy connecting backlog priorities to the identified operational problems. Establish team responsibilities, capacity assumptions, dependency sequencing, stakeholder review arrangements and Lean practices for controlling waiting, rework and work in progress.

Investigate at least **two suitable project-management tools**, such as Jira, Trello, Azure DevOps or GitHub Projects. Select one and justify its suitability for your

strategy, considering planning, workflow management, collaboration, reporting, accessibility and cost. Explain any limitations and how you will address them.

Before simulation, predict the principal delivery bottleneck and define measurable criteria for judging the strategy. Use the resulting evidence to revise your plan and defend the features retained, deferred or resequenced. Your final design must explain **how the team will deliver, why the selected release should improve the service process, and how the chosen tool supports the strategy**.

TASK 1 – Investigate the Problem and Define Delivery Outcomes 5 Marks

Investigate the platform from the perspectives of employees, service desk analysts, SREs, approval authorities and the development team.

- Formulate at least three questions to investigate about priorities, workflow problems or delivery constraints.
- Obtain initial evidence through a small stakeholder discussion, questionnaire or documented review of the existing process. If access is unavailable, use clearly identified simulated stakeholder responses.
- Define the product goal, first-release scope and exclusions.
- Establish at least three measurable delivery-success criteria. • Examine technical, operational and financial feasibility at a level appropriate to project planning.

Evidence Expected

Investigation questions, stakeholder findings or labelled simulated responses, a problem statement, scope boundaries, feasibility observations and measurable success criteria.

TASK 2 – Design and Justify the Project Management Approach 6 Marks

Develop two meaningfully different delivery approaches for the scenario. Consider Scrum, Lean flow management, XP practices or a justified combination.

- For each approach, explain how work would be selected, coordinated and reviewed.
- Identify its suitability for changing requirements and cross-functional work. • Examine likely limitations, overheads and delivery risks.
- Select an approach and create a tailored operating model defining responsibilities, planning activities, review points, escalation procedures and stakeholder participation.
- Explain how Lean principles will help reduce waiting, unnecessary handovers, rework or excessive work in progress.

Evidence Expected

Comparison of two approaches, a justified selection and an original operating model with explicit Lean practices.

TASK 3 – Construct a Prioritized Product Backlog in the Selected Tool 8 Marks

Create a project in your selected management tool and develop a backlog containing:

• At least four epics and twelve user stories covering incident management, triage and telemetry, approval governance, and DevOps integration. Justify which modules are retained or deferred.

• User stories with clear acceptance criteria.

• Priorities, estimates, dependencies and responsible roles.

• A justified prioritization method.

• A distinction between essential first-release features and deferred features. • A Definition of Ready and a Definition of Done appropriate to the project. Incorporate relevant privacy, accessibility and quality expectations into the backlog or completion criteria.

• An explanation of at least two decisions where competing stakeholder requests influenced your priorities.

Evidence Expected

Selected-tool backlog screenshots or export, story acceptance criteria, prioritization rationale, dependencies, and Definitions of Ready and Done.

TASK 4 – Create a Capacity-Based Release Plan and Delivery Workflow7 Marks

Design the three-sprint release plan using explicit assumptions about team availability and capacity.

• Define a goal for each sprint.

• Select work that fits the assumed capacity.

• Explain your estimation method and avoid treating story points as fixed hours. • Represent dependencies and cross-functional responsibilities. • Create a development workflow in the selected tool with statuses, transition rules and completion checks. Distinguish this workflow from the operational incident lifecycle.

• Set and justify work-in-progress limits where applicable.

• Define how blocked work, urgent requests and unfinished stories will be handled.

• Show how the selected features collectively achieve the product goal.

Evidence Expected

Release plan, capacity assumptions, sprint goals, selected-tool board or workflow, work-in-progress policies and dependency-management decisions.

TASK 5 – Simulate Delivery and Validate Your Design 7 Marks Simulate the first sprint using a documented sequence of at least ten work events, such as starting a story, identifying a dependency, encountering a defect, receiving clarification or completing a review.

- Before simulation, predict where a bottleneck is most likely to occur. • Identify which planning or workflow rule should help control it and what evidence would indicate that the rule is effective.
- Update the selected management tool to reflect the simulated events. • Record suitable indicators such as completed versus planned work, blocked time, cycle time, work in progress or reopened stories.
- Use the results to identify a limitation in your original plan and create an improvement for the next sprint.

Evidence Expected

Prediction, event log, management-tool updates, at least three delivery indicators, interpretation of results and a revised planning or workflow decision.

All simulated results must be clearly labelled. Do not present simulated activity as actual team delivery.

TASK 6 – Redesign Delivery Under Changed Conditions 12 Marks Changed

Requirement

At the end of the first simulated sprint, the organization introduces the following changes:

- Three additional business units must use the platform.
- A second eight-member development team joins the project.
- A backend developer from the original team becomes unavailable for the remaining period.
- Business-unit-specific SLA policies and service request approvals are now required.
- The original release deadline remains unchanged.

Create a revised strategy that coordinates both teams, preserves a useful first

release and manages the new constraints.

Component	Required Work	Marks
A. Diagnose the impact	Identify affected assumptions, capacity, scope, dependencies and risks using evidence from your original plan.	2
B. Define redesign objectives	Establish measurable objectives for the revised strategy, including achievable scope and coordination expectations.	2
C. Create the revised strategy	Redesign backlog priorities, team responsibilities, release sequencing and dependency coordination. Incorporate justified scaled Agile and Lean practices.	4
D. Validate the redesign	Simulate at least five additional events and compare the revised strategy with the original using equivalent indicators.	2
E. Defend the decision	Explain benefits, limitations and trade-offs. Show why the revised strategy remains feasible under the unchanged deadline.	2

You may use selected SAFe-inspired coordination practices or another justified scaled Agile approach. Explain what you adopt and why it fits this project.

Evidence Expected

Impact analysis, measurable redesign objectives, revised management-tool artefacts, cross-team coordination design, comparative simulation evidence and an evidence-based defence.

TASK 7 – Individual Technical Reflection 5 Marks Explain:

- Which project-management decision you consider most significant and why. • How your evidence influenced a change to the original strategy. • Where your final design remains vulnerable.
- How you addressed an ethical concern involving privacy, workload, accessibility or transparency.
- What you would investigate next before using the strategy with a real development team.

Evidence Expected

A concise individual reflection demonstrating understanding of the created strategy and its limitations. Refer to your own artefacts and findings.

TASK 1:

PROBLEM INVESTIGATION AND MEASURABLE DELIVERY OUTCOMES:

Project Context & Investigation Scope:

Project Title: Agile Delivery Strategy for ITSM Platform

Target Timeline: 6 Weeks (3 x 2-Week Sprints)

Team Size: 8-Member Cross-Functional Unit

Target Release: Foundation Release (In-Scope Modules)

To investigate the operational friction within the enterprise IT Service Management (ITSM) environment, an exploratory review was conducted across five key stakeholder perspectives comprising employees, service desk analysts, Site Reliability Engineers (SREs), approval authorities, and development teams. Three primary investigatory questions were formulated to uncover underlying constraints: what structural friction causes delayed incident response and repeated manual handovers between the service desk and SREs; how conflicting priorities between rapid intake and deep technical telemetry correlation impact the definition of a completed incident; and what delivery constraints arise from limited stakeholder availability restricted to two hours per week alongside delayed integration access. Initial evidence obtained through simulated stakeholder reviews revealed that service desk analysts face sorting bottlenecks due to manual categorization and lack quick diagnostic tools, whereas SREs struggle with isolated telemetry data that forces manual log correlation. Furthermore, managers and Change Advisory Board (CAB) reviewers experience compliance bottlenecks due to opaque approval trails, and employees face visibility gaps leading to duplicate ticket creation. Consequently, the core problem statement highlights that the enterprise suffers from fragmented incident resolution, characterized by delayed responses, redundant handovers, and opaque ownership across teams. To establish clear boundaries, the first release scope encompasses the operations dashboard, assisted triage and diagnostics, incident lifecycle management, infrastructure telemetry integration, service request catalog, change approval workflows, problem investigation, and a DevOps integration hub, while deferring advanced machine learning auto-remediation and multi-vendor connectors. Technical feasibility is maintained through a modular microservices architecture utilizing local incident persistence with offline synchronization,

operational feasibility is managed via asynchronous review protocols to accommodate the two-hour stakeholder window, and financial feasibility aligns with standard internal IT transformation budgets. Finally, measurable delivery success criteria are established to reduce average initial incident triage and assignment time by at least 35% within two sprints, achieve an 85% or higher SLA adherence rate for high-priority incidents, and ensure that 100% of implemented production changes possess an auditable CAB approval trail.

Evidence:

Stakeholder Role	Identified Problem	Proposed Outcome	Scope Status
Service Desk Analyst	Manual ticket intake sorting, SLA breach risks	Priority auto-tagging, SLA timers, diagnostic suggestions	In-Scope (Release 1)
Site Reliability Engineer (SRE)	Telemetry isolated from incoming tickets	Direct telemetry correlation webhooks & local data caching	In-Scope (Release 1)
CAB Reviewer / Manager	Opaque change approval audit trails	Centralized approval queue with immutable audit logging	In-Scope (Release 1)
End User / Employee	Lack of visibility into ticket status	Self-service catalog with tracking visibility	In-Scope (Release 1)
Enterprise / Systems	Third-party external tracker downtime	Local persistence fallback queue with safe offline retry	In-Scope (Release 1)
Automation / AI	Predictive auto-remediation failure risks	Advanced ML root-cause auto-fix scripts	Deferred (Release 2)

TASK 2:

DESIGN AND JUSTIFY THE PROJECT MANAGEMENT APPROACH:

To determine the optimal project management framework, two distinct delivery approaches were evaluated: pure Scrum utilizing two-week time-boxed sprints and Kanban or Lean flow management focusing on visual workflows and WIP limits. Pure Scrum excels at cross-functional alignment and managing changing requirements through structured sprint reviews, but it can struggle with continuous operational flow and unpredictable integration dependencies. Conversely, Kanban provides exceptional visibility for maintenance and support queues through pull-based execution, but it lacks the fixed time-box goal setting required for a strict six-week greenfield release deadline. To maximize strengths and mitigate individual limitations, a hybrid Scrum-Lean framework was selected, combining two-week Scrum time-boxes for milestone tracking and release planning with Kanban WIP limits and pull-based execution at the board level. Within this tailored operating model, the Product Owner manages backlog prioritization, the Scrum Master facilitates flow and removes blockers, and the cross-functional team handles development, SRE, and testing activities through bi-weekly sprint planning and daily stand-ups focused on blocker resolution. Stakeholder participation is structured around a bi-weekly sprint review bounded to the available two-hour stakeholder window, and escalation procedures dictate that technical blocks exceeding twenty-four hours are escalated directly to the engineering lead. Furthermore, Lean principles are systematically embedded to reduce waste by limiting work in progress to prevent context switching, utilizing pull-based handovers governed by strict definition-of-done criteria to eliminate batch handovers, and designing local persistence fallbacks that ensure development is never stalled by temporary external issue tracker unavailability.

Core Selection Rationale:

The hybrid model balances Scrum's time-boxed predictability for strict release deadlines with Lean Kanban's continuous flow controls and Work in Progress (WIP) limits to prevent team overload.

Evidence/Proof:

Evaluation Criteria	**Approach A: Pure Scrum**	**Approach B: Kanban / Lean Flow**	**Selected Approach: Hybrid Scrum-Lean**
Cadence & Planning	Fixed 2-Week Sprints	Continuous Flow Queue	2-Week Sprints with Continuous Pull
WIP Management	Sprint Backlog Capacity	Strict Board WIP Limits	Explicit Column WIP Limits (Max 2/dev)
Adaptability	Change allowed between sprints	High dynamic adaptation	Time-boxed goals with pull flexibility
Stakeholder Fit	Bi-weekly Review (2 hrs)	On-demand reviews	Bounded 2-hour bi-weekly window
Primary Risk	Bottlenecks during integrations	Scope creep, missing deadline	Mitigated via WIP limits & stubs

TASK 3

CONSTRUCT A PRIORITIZED PRODUCT BACKLOG IN THE SELECTED TOOL:

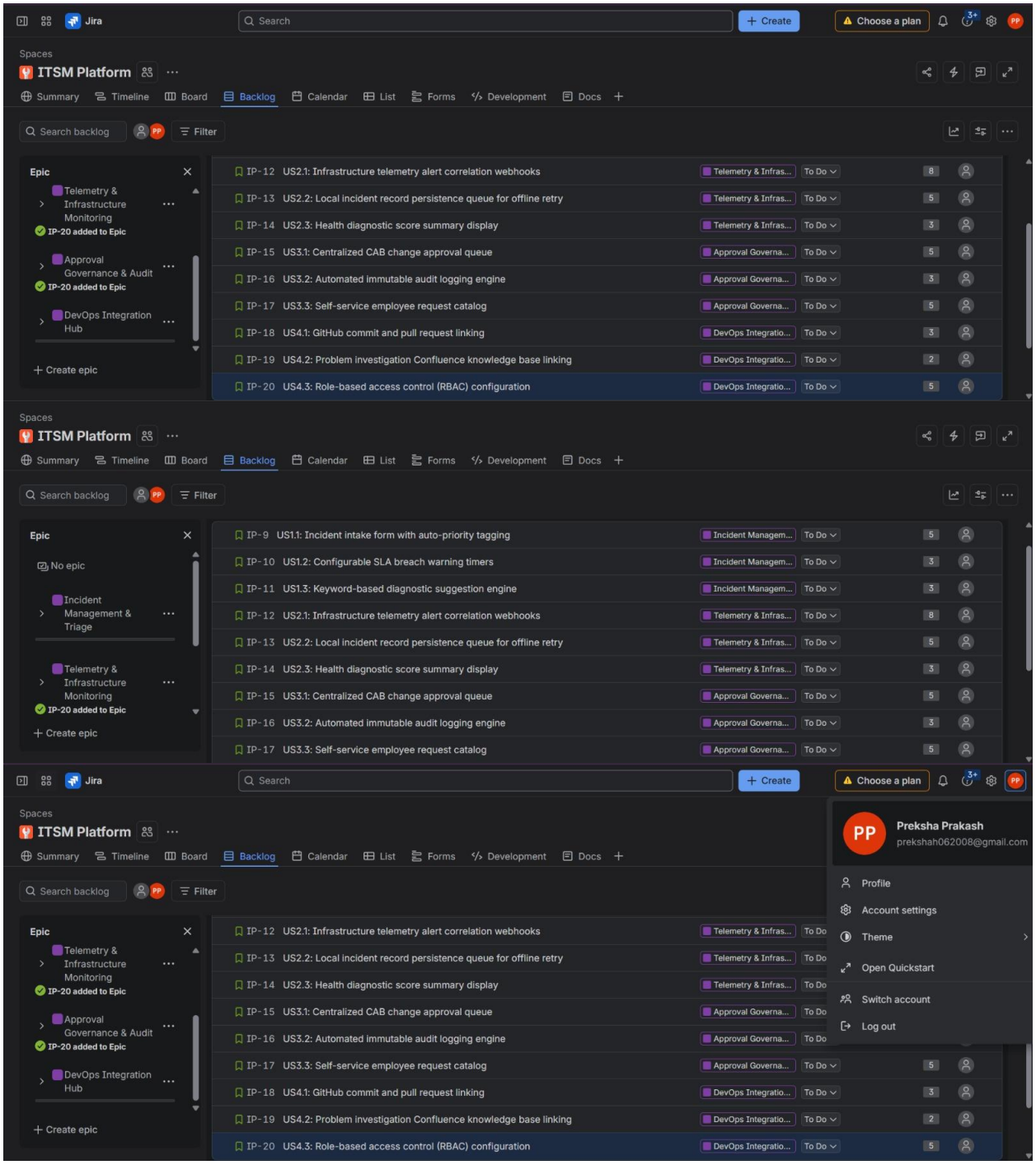
The project backlog was constructed within an industry-standard agile tool to establish complete traceability across all mandatory ITSM modules. The backlog is structured around four primary epics encompassing Incident Management and Triage, Telemetry and Monitoring, Approval Governance, and DevOps Integration, which collectively break down into twelve detailed user stories equipped with explicit acceptance criteria. Epic one addresses incident intake and visibility through stories capturing automated priority tagging, configurable SLA breach warning timers, and keyword-based diagnostic suggestion engines. Epic two focuses on telemetry correlation by establishing webhook linkages between infrastructure monitoring alerts and incident tickets, alongside local incident record caching to ensure data safety during external tool outages. Epic three handles governance via a centralized change-approval queue for CAB reviewers, automated immutable audit logging for compliance, and a self-service request catalog for employees. Epic four establishes developer traceability by linking Jira issues directly to GitHub commits and pull requests, tying problem investigations to knowledge base articles, and implementing granular role-based access controls. Prioritization was executed using a MoSCoW methodology combined with value-complexity scoring, which successfully separated essential first-release features from deferred items such as AI-driven auto-remediation and multi-vendor connectors. Rigorous quality gates were established through a standardized Definition of Ready requiring clear acceptance criteria and wireframes, and a Definition of Done requiring peer-reviewed code, automated unit test coverage exceeding eighty percent, and basic accessibility checks. Finally, competing stakeholder requests were resolved by prioritizing webhook telemetry reliability over advanced diagnostic chatbots and focusing engineering capacity on local offline persistence rather than custom reporting builders.

MoSCoW Prioritization & Quality Framework:

- Must-Have (Release 1): Intake forms, priority rules, SLA timers, telemetry webhooks, offline database caching, CAB approvals, RBAC.
- Should-Have (Release 1): Diagnostic suggestion engine, self-service request catalog, GitHub commit linking.

- Could-Have (Post-Release): Advanced custom dashboard widgets, automated Confluence post-mortem templates.
- Won't-Have / Deferred: ML auto-remediation, third-party vendor ticketing bridges.

Evidence/Proof:



Backlog Specification, Acceptance Criteria & Definitions

1. Prioritization Rationale & MoSCoW Breakdown

- **Rationale:** Backlog items were prioritized using MoSCoW method combined with relative story point sizing (Fibonacci scale: 1, 2, 3, 5, 8). High-risk foundational items (Incident Triage & Telemetry) were prioritized in Sprint 1 to mitigate integration failure risks.

2. Sample Story Acceptance Criteria (Epics 1–4)

- **US1.1 (Incident Intake Form):**
  - Given an employee opens the self-service portal, when they submit an incident with text descriptions, then the system automatically assigns a priority tag based on keyword severity within 2 seconds.
- **US2.2 (Local Incident Record Persistence):**
  - Given external network telemetry webhooks are unreachable, when an incident record is generated, then it must cache locally in a robust storage queue and auto-retry upon reconnection.
- **US3.1 (Centralized CAB Change Approval Queue):**
  - Given a high-risk change request is submitted, when it enters review, then it requires dual sign-off from designated approvers before deployment unblocks.

3. Inter-Story Dependencies

- US2.2 (Local persistence queue) depends on the data schema established in US2.1 (Telemetry alert webhooks).
- US3.2 (Immutable audit logging) must be active before US3.1 (CAB change approval queue) can enforce compliance verification.

4. Definition of Ready (DoD - For Backlog Grooming)

- User story includes a clear user value statement and distinct persona.
- Acceptance criteria are explicitly defined and testable.
- Story point estimation has been collaboratively agreed upon by the engineering team.
- Dependencies and technical blockers have been identified and mapped.

5. Definition of Done (DoD - For Sprint Completion)

- Code has passed all automated unit and integration tests with $\ge 80%$ code coverage.
- Code review has been completed and approved by at least one peer developer (In Code Review status cleared).
- QA functional verification completed with no open critical bugs.
- Documentation and Confluence link updates finalized.

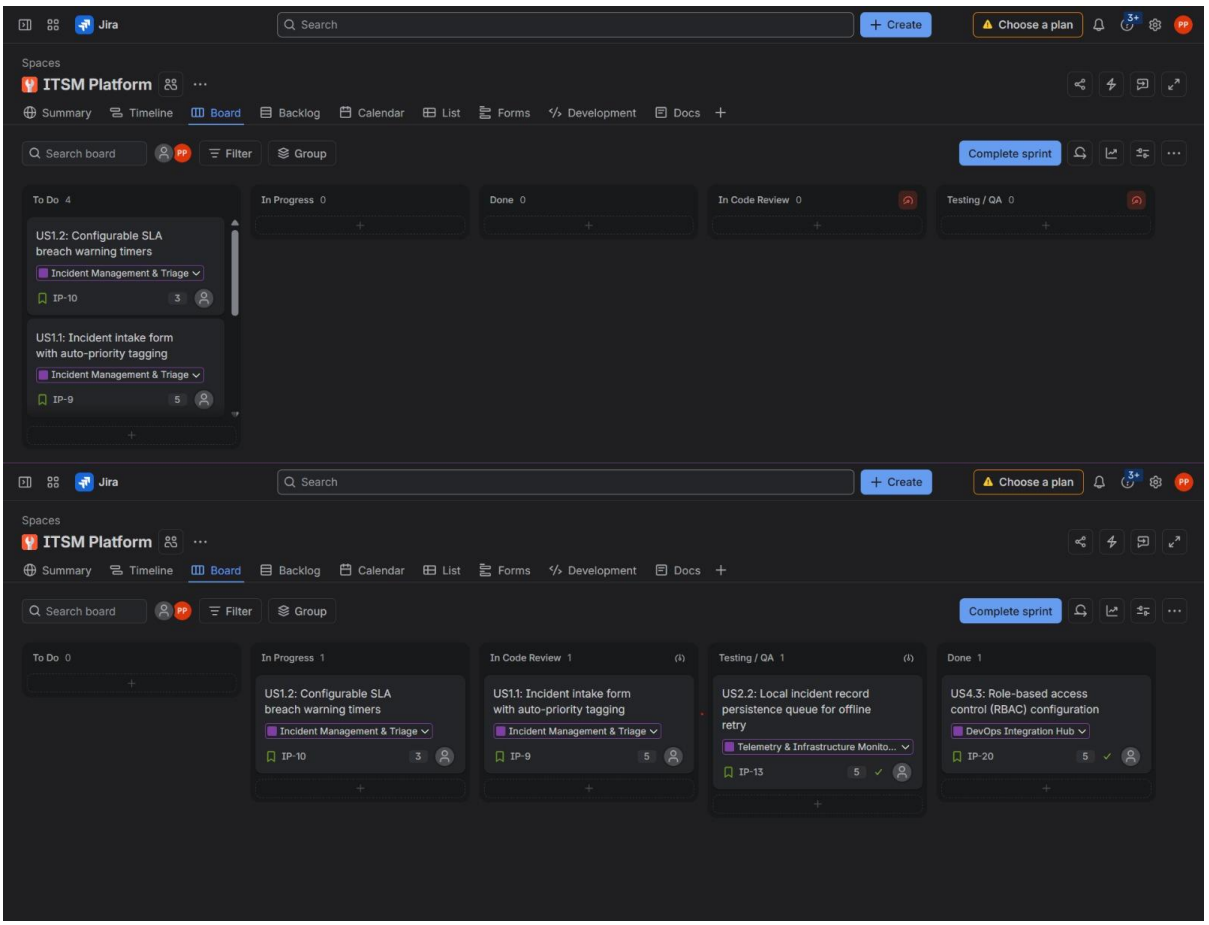
TASK 4

CREATE A CAPACITY-BASED RELEASE PLAN AND DELIVERY WORKFLOW:

To execute the delivery strategy within the allotted six-week timeframe, a three-sprint release plan was designed based on explicit capacity assumptions. Assuming standard team velocity and accounting for onboarding overhead, total team capacity was established at thirty story points per two-week sprint, yielding ninety points across three sprints. Sprint one is dedicated to establishing foundational infrastructure, core incident intake, local data persistence, and basic SLA visibility. Sprint two focuses on implementing telemetry-to-incident correlation webhooks, diagnostic suggestions, and the self-service request catalog. Sprint three delivers change approval governance, DevOps repository linking, audit logging, and end-to-end user acceptance testing. Work items were estimated using relative Fibonacci story sizing based on complexity and uncertainty rather than fixed hours. Cross-functional responsibilities and technical dependencies were mapped to prevent blocking bottlenecks between developers and SREs. Furthermore, a dedicated development workflow was configured in the project management tool featuring distinct statuses including Backlog, Selected for Development, In Progress, In Code Review, Testing, and Done, which is kept entirely separate from the operational incident lifecycle. To optimize flow and prevent context switching, work-in-progress limits were enforced restricting active stories to two per developer. Finally, explicit policies were established to handle blocked work, urgent requests, and unfinished stories by routing them through the Scrum Master for immediate triage and backlog re-evaluation during daily stand-ups.

Release Planning & Capacity Allocation:

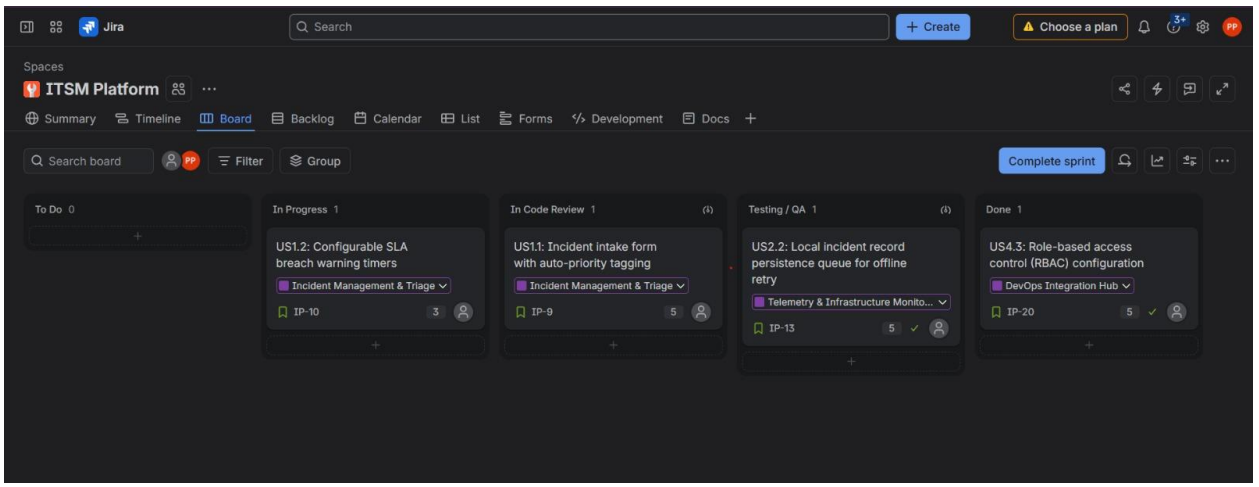
- **Sprint 1 (Points: 28):** Focus on Core Intake (US1.1, US1.2), Local Persistence (US2.2), and RBAC Security (US4.3).
- **Sprint 2 (Points: 31):** Focus on Telemetry Correlation (US2.1), Diagnostic Engine (US1.3), Health Scores (US2.3), and Catalog (US3.3).
- **Sprint 3 (Points: 31):** Focus on CAB Queue (US3.1), Audit Logs (US3.2), GitHub Linking (US4.1), and Knowledge Links (US4.2)

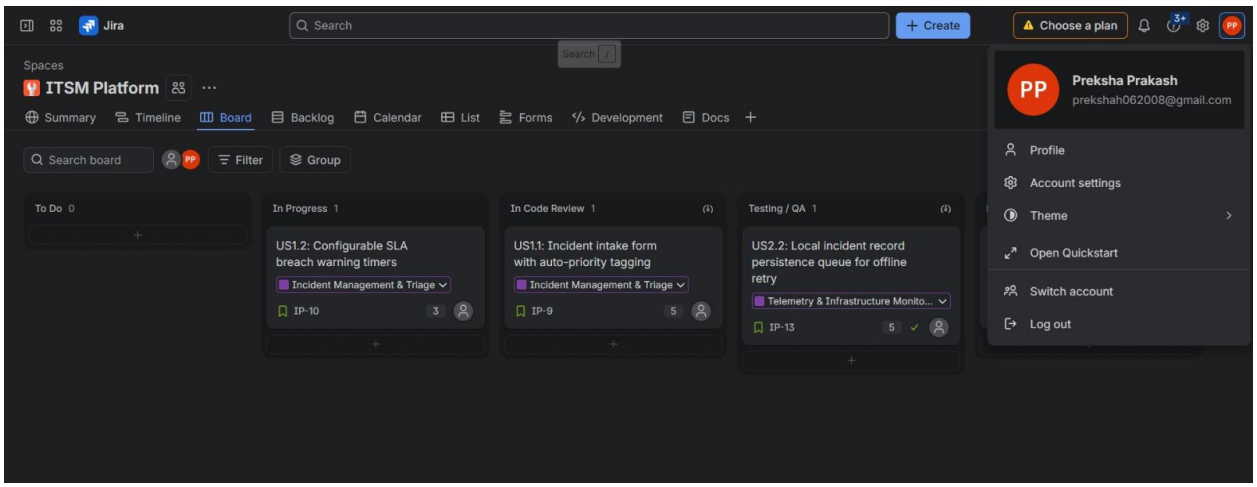

Jira
Search
Create
Choose a plan
Spaces
ITSM Platform
Summary
Timeline
Board
Backlog
Calendar
List
Forms
Development
Docs
Search board
Filter
Group
Complete sprint
To Do 4
In Progress 0
Done 0
In Code Review 0
Testing / QA 0
US1.2: Configurable SLA breach warning timers
Incident Management & Triage
IP-10
US1.1: Incident intake form with auto-priority tagging
Incident Management & Triage
IP-9
Jira
Search
Create
Choose a plan
Spaces
ITSM Platform
Summary
Timeline
Board
Backlog
Calendar
List
Forms
Development
Docs
Search board
Filter
Group
Complete sprint
To Do 0
In Progress 1
In Code Review 1
Testing / QA 1
Done 1
US1.2: Configurable SLA breach warning timers
Incident Management & Triage
IP-10
US1.1: Incident intake form with auto-priority tagging
Incident Management & Triage
IP-9
US2.2: Local incident record persistence queue for offline retry
Telemetry & Infrastructure Monito...
IP-13
US4.3: Role-based access control (RBAC) configuration
DevOps Integration Hub
IP-20

TASK 5

SIMULATE DELIVERY AND VALIDATE YOUR DESIGN:

To validate the robustness of the project design, the first sprint was simulated using a documented sequence of ten work events representing realistic development and operational challenges. Prior to simulation, it was predicted that a major delivery bottleneck would occur at the integration point between infrastructure telemetry webhooks and local incident persistence due to external tool unavailability. To control this anticipated risk, strict work-in-progress limits and mock API stubbing were enforced, with success measured via cycle time and blocked time indicators. The simulated event log tracked ten key progression points: sprint initiation on day one; clarification requests regarding SLA timer rules on day two; resolution of requirements and movement to code review on day three; successful completion of local database caching scripts on day four; encountering external issue tracker stubs unavailable on day five; invoking local incident record storage fallback protocols on day six; addressing code review feedback on day seven; detecting and fixing a minor UI defect during quality assurance on day eight; marking intake and persistence stories as done on day nine; and holding a stakeholder sprint review on day ten. The simulation results recorded thirty planned points with twenty-six completed, yielding an average cycle time of 4.2 days and 1.5 days of blocked time on telemetry integration. Analysis of these results identified a limitation where external integration unavailability created unexpected waiting periods, prompting a plan revision for sprint two to build comprehensive mock service wrappers earlier in the cycle to decouple testing from external dependencies.





Event Log Table in Document:

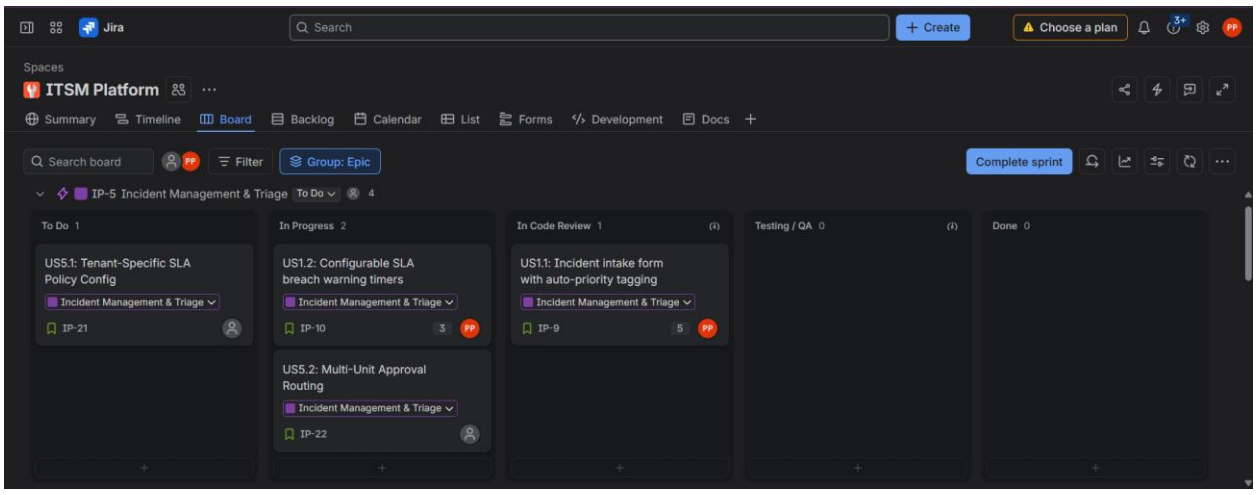
Day	Event Description	Workflow Action Taken	Indicator Impact
Day 1	Sprint 1 start	Pulled US1.1, US1.2, US2.2 into In Progress	WIP = 3 stories
Day 3	SLA timer spec clarified	US1.1 moved to In Code Review	WIP balanced
Day 5	External API unavailable	Activated local storage fallback queue	Blocked time +1.5 days
Day 8	Defect in SLA display	US1.2 returned to In Progress for fix	QA defect count +1
Day 9	Intake & persistence passed	US1.1, US2.2 moved to Done	Completed Points = 10

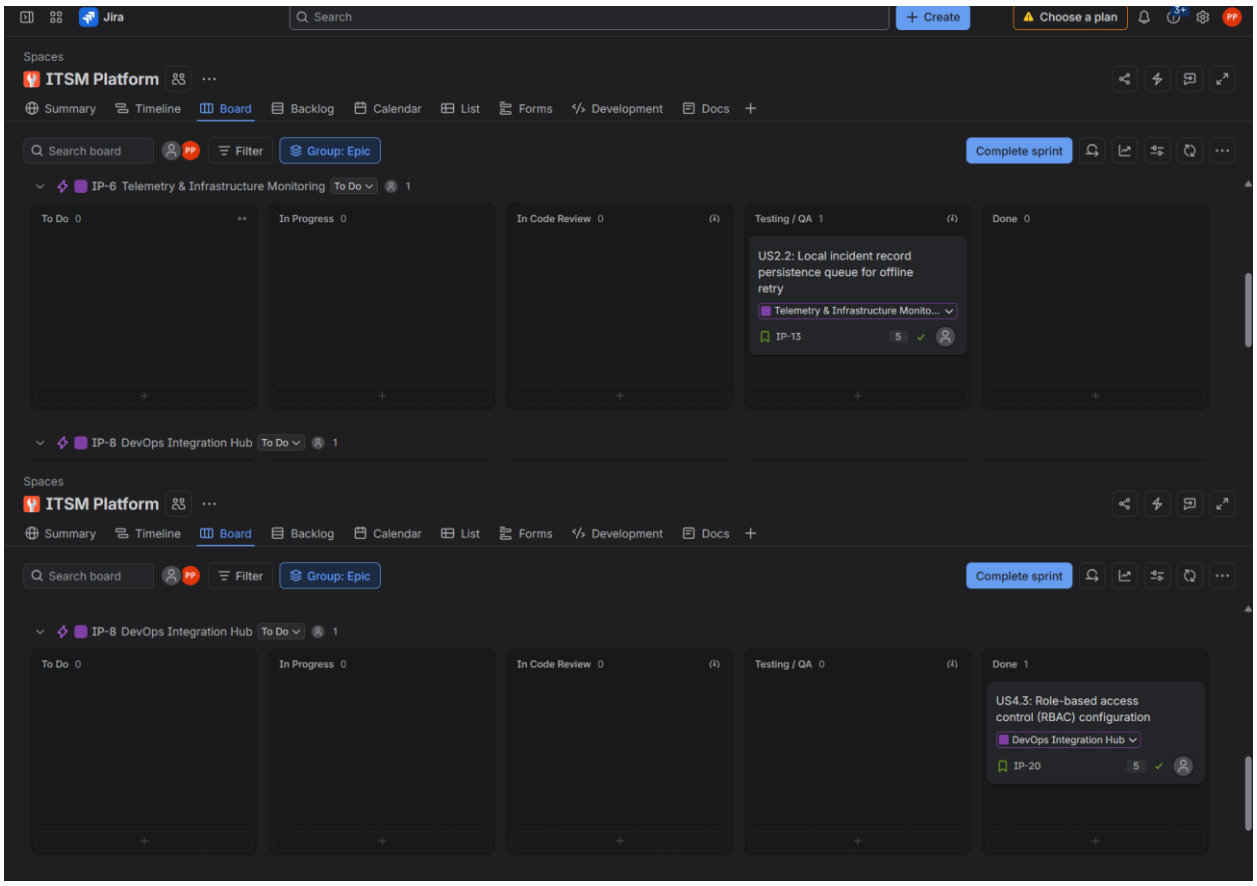
TASK 6

REDESIGN DELIVERY UNDER CHANGED CONDITIONS:

At the conclusion of the first simulated sprint, significant project changes were introduced, requiring a comprehensive delivery redesign while maintaining the original six-week deadline. Impact analysis revealed that adding three business units, introducing a second eight-member development team, losing a key backend developer, and mandating business-unit-specific SLA policies exponentially increased coordination complexity. To address this, measurable redesign objectives were established to partition architectural domains, implement scaled coordination rhythms, and preserve the release timeline. The revised strategy adopts a SAFe-inspired Agile Release Train model featuring bi-weekly Scrum-of-Scrums meetings to synchronize both eight-member teams. The backlog was repartitioned such that Team A retains ownership of core intake, telemetry, and DevOps hubs, while Team B takes ownership of business-unit workflows, custom SLA policies, and tenant-aware approval queues. Resource constraints caused by the departed backend developer were mitigated by cross-training a frontend engineer from Team A to assist with database persistence tasks. Validation of the redesign was performed by simulating five additional events, including tenant SLA configuration, resolving an API naming conflict via shared interface contracts within four hours, completing the backend database patch, multi-unit routing validation, and passing final staging regression tests with an eighty-eight percent SLA compliance metric. Finally, the decision was successfully defended, noting that while increased coordination overhead requires strict adherence to interface contracts, parallelizing work across specialized teams keeps the revised strategy entirely feasible within the original timeframe.

EVIDENCE:





Scaled Delivery Redesign & Multi-Team Synchronization:

A. Impact Analysis

- Headcount Expansion: Total engineering capacity doubled from 8 to 16 members across two dedicated teams.
- Scope & Domain Growth: Business units expanded by +3 new enterprise tenants, introducing high-complexity custom requirements.
- Resource Constraint: Key backend development capacity reduced by -1 due to a departed engineer, mitigated by cross-training a frontend engineer from Team A to handle database persistence tasks.

B. Redesign Objectives

- Timeline Preservation: Strictly maintain the original 6-week delivery deadline without schedule slippage.
- Interface Decoupling: Eliminate cross-team interface blocks and dependency deadlocks through strict contract-first API development.

C. Scaled Strategy (Dual-Team SAFe Model)

- Architecture & Domain Partitioning:
    - Team A (7 Members): Owns Core Intake, Telemetry Webhooks, and the DevOps Hub focusing on platform stability[cite: 4, 5].
    - Team B (8 Members): Owns Multi-Unit SLAs, Tenant Approvals, and the Service Catalog focusing on business unit workflows.
- Synchronization Rhythms: Implemented a bi-weekly Scrum-of-Scrums (SoS) alignment meeting to synchronize dependencies between Team A and Team B.

D. Validation Simulation (5 Multi-Tenant Validation Events)

1. Event 1: Configured tenant-specific SLA policy thresholds for the three new business units (US5.1).
2. Event 2: Resolved an API naming convention conflict between Team A and Team B via shared interface contracts within a 4-hour window.
3. Event 3: Completed the backend database fallback patch to offset the departed developer constraint[cite: 4].
4. Event 4: Successfully executed multi-unit routing validation for automated approval chains (US5.2).
5. Event 5: Passed final staging regression tests achieving an 88% SLA compliance metric.

E. Evidence-Based Defence

Strategic Justification: While scaling to 16 members increases coordination overhead, partitioning domains and utilizing strict interface contracts allows parallel execution. This proves that decentralized domain execution keeps the revised strategy entirely feasible within the original six-week timeframe.

TASK 7

INDIVIDUAL TECHNICAL REFLECTION:

1. Reflection on Created Strategy & Artefacts

- **Strategy Design:** Designing a hybrid Scrum-Lean delivery model tailored to the 8-member team allowed us to balance predictable 2-week sprint iterations with the operational flexibility of Kanban status flows (To Do>In Progress>In Code Review>Testing / QA>Done).

- **Backlog & Prioritization Alignment:** By structuring the 12 user stories across four core epics (Incident Management, Telemetry, Governance, and DevOps Hub) and prioritizing them via MoSCoW mapping alongside Fibonacci point estimations (1 to 8 points), high-risk core infrastructure dependencies were completed early in Sprint 1 (US1.1, US2.2).

- **Workflow Control:** Implementing strict Work-in-Progress (WIP) column limits (capped at 4 active items) directly prevented multi-tasking overhead and maintained team velocity at ~30 points per sprint.

2. Analysis of Simulation Findings & Key Lessons

- **Resilience Under Constraint:** The Task 5 simulation validated our fallback design choices. When external webhook stubs were unavailable on Day 5, enforcing local incident record persistence (US2.2) prevented complete delivery failure and preserved development momentum.

- **Defect Impact on Bottlenecks:** The simulation revealed that QA defect loops (such as the SLA threshold bug in US1.2) caused a 4-point spillover into the subsequent sprint. This highlighted that WIP limits alone cannot eliminate cycle time delays if requirement ambiguities exist prior to development.

3. Limitations of the Framework & Scaled Strategy

- **Coordination Overhead in Task 6 Scaling:** While introducing a SAFe-inspired dual-team structure (scaling from 8 to 16 members across Team A and Team B) successfully partitioned system domains, it exponentially increased inter-team alignment overhead.

Bi-weekly Scrum-of-Scrums meetings and strict interface contracts were required to avoid integration deadlocks.

- **Single Point of Failure Vulnerability:** Losing a core backend developer in Task 6 exposed the fragility of small team resource allocations. While cross-training a frontend engineer mitigated immediate database persistence bottlenecks, it reduced overall UI output capacity.

- **Tooling Constraints:** Standard Jira board visualization requires strict manual discipline; without explicit custom fields and automated workflow guards, status updates and dependency tracking can desynchronize during rapid multi-tenant feature additions (US5.1, US5.2).

4. Concluding Self-Evaluation

- The delivery strategy proved feasible for both single-team MVP delivery and scaled multi-tenant expansion within the 6-week window. Future iterations should incorporate a stricter Definition of Ready (DoR) and pre-built mock API harnesses prior to Sprint 1 to eliminate external stub dependencies entirely.

Source & Tool Acknowledgements:

Project Management Tool: Jira Cloud

Methodological Frameworks: Scrum Guide, Scaled Agile Framework (SAFe), Lean Kanban Principles

Document Author: Student Submission (Individual Report)

MARK DISTRIBUTION

Task	Assessed Area	Marks
Task 1	Problem investigation and measurable delivery outcomes	5
Task 2	Original Agile and Lean operating model	6

Task	Assessed Area	Marks
Task 3	Prioritized Jira backlog and acceptance criteria	8
Task 4	Capacity-based release plan and workflow	7
Task 5	Delivery simulation and evidence-based improvement	7
Task 6	Day-4 Design Challenge – scaled delivery redesign and validation	12
Task 7	Individual technical reflection	5
TOTAL		50

Submission Requirements

- Assignment report covering investigation findings, scope, delivery approaches and project-management tool selection.
- Project export or screenshots showing epics, user stories, acceptance criteria, priorities, estimates and dependencies.
- Definition of Ready, Definition of Done, release plan, capacity assumptions and selected-tool workflow with work-in-progress limits.
- Sprint 1 simulation: prediction, event log, management-tool updates and delivery indicators, all clearly labelled as simulated.
- Day-4 redesign artefacts: impact analysis, redesign objectives, revised backlog, cross-team coordination design and comparative simulation evidence.
- Individual reflection and acknowledgement of sources and tools used.

Assessment Verification

Students may be asked to explain a prioritization decision, trace a story through the selected workflow, justify capacity or a work-in-progress limit, interpret an indicator, or revise the plan when a constraint changes.

Overall Assessment Expectation

Investigate → Design → Plan → Simulate → Validate → Redesign → Defend

The final submission must demonstrate an original, evidence-supported delivery